

Compte rendu n°1

Tp Tests de Logiciels

Nom : Rayen Bouyahia

Classe : L2CS01

1. Introduction

Ce TP porte sur la mise en place de tests unitaires dans un projet Java, en utilisant JUnit 5 comme framework de test. L'objectif est de comprendre comment structurer des tests automatiques pour vérifier le comportement d'une classe métier (ici la classe Rational), tout en maîtrisant les mécanismes de base offerts par JUnit 5.

Les notions abordées dans ce TP sont :

- La création d'un projet Java avec dossier de test sous Eclipse.
- L'écriture de tests unitaires avec les assertions JUnit 5.
- La gestion du temps d'exécution via l'annotation @Timeout.
- Le cycle de vie des tests avec les annotations Before / After.

2. Création du projet + premier test

2.1 Présentation

La première étape consiste à créer un projet Java sous Eclipse et à y ajouter un dossier de test. Une classe de test JUnit 5 est ensuite générée et configurée automatiquement avec la bibliothèque JUnit.

2.2 La classe Rational

La classe Rational représente un nombre rationnel (numérateur / dénominateur). Elle possède les fonctionnalités suivantes :

- Un constructeur qui lance une IllegalArgumentException si le dénominateur est zéro.
- Une méthode add(Rational other) pour additionner deux rationnels.
- Une simplification automatique via le PGCD (méthode privée gcd).
- Les méthodes equals() et toString() redéfinies.

Exemple d'implémentation :

```
public Rational(int numerator, int denominator) {
    if (denominator == 0)
        throw new IllegalArgumentException("Le dénominateur ne peut pas être zéro");
    this.numerator = numerator;
    this.denominator = denominator;
    simplify();
}
```

2.3 Premier test unitaire

Un test unitaire suit le patron classique GIVEN / WHEN / THEN. Voici le test d'addition de

deux rationnels :

```
@Test
void testAddition() {
    // GIVEN
    Rational r1 = new Rational(1, 3);
    Rational r2 = new Rational(2, 1);
    // WHEN
    Rational result = r1.add(r2);
    // THEN
    assertEquals(7, result.getNumerator(), "Numérateur incorrect");
    assertEquals(3, result.getDenominator(), "Dénominateur incorrect");
}
```

La méthode `assertEquals` vérifie l'égalité entre la valeur attendue et la valeur obtenue. En cas d'échec, le message fourni en troisième paramètre est affiché.

3. Classe TimeoutTest (temps d'exécution)

3.1 Présentation

L'annotation `@Timeout` permet d'imposer une durée maximale d'exécution à un test. Si le test dépasse ce seuil, il échoue automatiquement avec une erreur de type `TimeoutException`.

Cela est utile pour détecter :

- Des boucles infinies ou des traitements trop longs.
- Des appels réseau ou des accès disque non optimisés.
- Tout comportement bloquant dans une application.

3.2 Exemple

```
@Test
@Timeout(value = 10, unit = TimeUnit.MILLISECONDS)
void test() throws Exception {
    Thread.sleep(1000); // dépasse le temps autorisé
}
```

Dans cet exemple, le test est limité à 10 millisecondes. Comme `Thread.sleep(1000)` attend 1000 ms, le test échoue avec 1 erreur et 0 échec (Errors: 1, Failures: 0).

4. Cycle de vie des tests (Before / After)

4.1 Présentation

JUnit 5 propose un ensemble d'annotations permettant de contrôler l'exécution avant et après les tests. Ces méthodes de cycle de vie facilitent l'initialisation et le nettoyage des ressources utilisées pendant les tests.

4.2 Les annotations disponibles

Annotation	Moment d'exécution	Remarque
<code>@BeforeAll</code>	Une seule fois avant tous les tests	Méthode statique

@BeforeEach	Avant chaque test	Initialisation par test
@AfterEach	Après chaque test	Nettoyage par test
@AfterAll	Une seule fois après tous les tests	Méthode statique

4.3 Exemple et ordre d'exécution

```
@BeforeAll    static void setUpBeforeClass() { /* exécuté une fois */ }
@BeforeEach  void setUp() { /* avant chaque test */ }
@Test        void test1() { /* test 1 */ }
@AfterEach   void tearDown() { /* après chaque test */ }
@AfterAll    static void tearDownAfterClass() { /* exécuté une fois */ }
```

L'ordre d'exécution observé dans la console pour 3 tests est :

```
@BeforeAll    →      @BeforeEach    →      test1      →      @AfterEach
      →      @BeforeEach    →      test2      →      @AfterEach
      → @BeforeEach → test3 → @AfterEach → @AfterAll
```

5. Bilan & Points clés

Ce TP a permis de maîtriser les bases des tests unitaires avec JUnit 5 :

- Création d'un projet de test Eclipse avec JUnit 5 et utilisation d'assertEquals.
- Détection des traitements lents grâce à @Timeout.
- Gestion du cycle de vie des tests avec @BeforeAll, @BeforeEach, @AfterEach et @AfterAll.
- Séparation claire entre la logique de test et l'initialisation / nettoyage des ressources.